Vignette: Lobster Growth Model

Authors: G. Nesslage & M. Wilberg

Environment: R Template Model Builder (RTMB)

1. Executive Summary & Theoretical Background

Modeling crustacean growth presents unique mathematical and statistical challenges relative to finfish. Because American lobsters (*Homarus americanus*) grow discretely by shedding their exoskeleton (ecdysis), continuous growth models such as the standard von Bertalanffy growth function are inadequate. Instead, stock assessments partition crustacean growth into two distinct processes:

1. **Molt Probability:** The length-dependent likelihood that an individual of a given carapace length (CL) will undergo a molt during a specified transition period.
2. **Molt Increment:** The expected linear size increase (in mm CL) achieved conditional on a molt occurring.

Mark-recapture data derived from field tagging programs provide the empirical basis for estimating these growth parameters. However, estimation is confounded by size-dependent tag reporting selectivity and differential tag shedding during ecdysis. This vignette demonstrates how to use the Lobster Growth Model in R using RTMB to estimate growth model parameters, construct Growth Transition Matrices (GTMs), and simulate multi-year population projections for comparison with that of the 2025 Atlantic States Marine Fisheries Commission (ASMFC) benchmark stock assessment.

2. Required R Packages and Workspace Setup

The implementation leverages RTMB for C++ automatic differentiation within R, combined with other packages for data cleaning and plotting.

```
# Clear environment and graphics
rm(list = ls(all = TRUE))
graphics.off()

# Load primary libraries
library(RTMB)         # Automatic differentiation & likelihood construction
library(dplyr)        # Data manipulation
```

```r
library(tidyr)       # Data restructuring
library(ggplot2)     # Graphics
library(lubridate)   # Date conversions
library(readxl)      # Excel data import
library(patchwork)   # Graphical combinations
library(gridExtra)   # Grid layout helpers
library(cowplot)     # Plot themes
```

3. Data Structure and Preprocessing

Tag-recapture datasets are maintained by the ASMFC for both American Lobster stock units—Gulf of Maine / Georges Bank (**GOMGBK**) and Southern New England (**SNE**). Raw observations must be filtered to eliminate short-term recaptures (unlikely to have undergone ecdysis) and extreme measurement anomalies.

Data Variable	Description	Filtering Criteria	Rationale
TagCL	Carapace length at release (mm)	TagCL ≥ 53 mm	Focuses estimation on recruit and legal size classes used in the 2025 assessment.
DaysAtLarge	Duration between tag release and recapture	DaysAtLarge > 21 days	Removes immediate recaptures before molting windows.
SizeDiff	Observed change in carapace length (mm)	SizeDiff < 20 mm	Excludes severe measurement errors or double-molts.
Source	Field sampling program identifier	Filter specific sources	Excludes non-representative sampling programs (e.g. TLC, DFO).

```r
# Example preprocessing for GOMGBK and SNE datasets
GOMGBKdat <- read.csv("AllTagDatGOMGBKclgNEWAnnual_3.csv") %>%
  mutate(rate = SizeDiff / TagCL, SU = "GOMGBK") %>%
  filter(SizeDiff < 20 & TagCL >= 53 & DaysAtLarge > 21) %>%
```

```
  filter(!(Source %in% c("TLC", "DFO", "NHSCUBA", "NHTbar")))

SNEdat <- read.csv("AllTagDatSNEclgNEWAnnual_2.csv") %>%
  mutate(rate = SizeDiff / TagCL, SU = "SNE") %>%
  filter(SizeDiff < 20 & TagCL >= 53 & DaysAtLarge > 21) %>%
  filter(!(Source %in% c("TLC")))
```

4. Mathematical Formulation of the Growth Model

4.1 Selectivity and Tag Reporting Probability

When tag reporting rates vary with lobster size, observed growth probabilities are biased. Tag reporting selectivity is parameterized using a logistic function of length:

```
sel <- function(x, int, slope) {
  return(exp(int + x * slope) / (1 + exp(int + x * slope)))
}
```

4.2 Molt Probability Model

The probability of molting is modeled via a generalized linear model with a logistic link function incorporating initial carapace length, days at large, and temperature / growing degree days:

$Prob(Growth) = e^{(\beta 0+\beta 1\boldsymbol{CL}+\beta 2\boldsymbol{DAL}+\beta 3\boldsymbol{Temp})}/(1+e^{(\beta 0+\beta 1\boldsymbol{CL}+\beta 2\boldsymbol{DAL}+\beta 3\boldsymbol{Temp})})$

4.3 Molt Increment Model

The expected growth increment conditional on molting is a linear model based on length at tagging, days at large, and temperature:

$Growth\ incremement = \gamma 0+\gamma\boldsymbol{CL}+\gamma 2\boldsymbol{DAL}+\gamma 3\boldsymbol{Temp}$

The incremental distribution can follow either a normal distribution with standard deviation proportional to the mean or a lognormal distribution with constant log-scale standard deviation.

4.4 Bayesian Tag Loss Correction

To account for differential tag shedding during ecdysis, raw observed molting probabilities are corrected using Bayes' Theorem:

```
correct_G_prob <- function(obs_G_prob, tag_reten_molt, tag_reten_nomolt) {
  p1 <- obs_G_prob * tag_reten_nomolt / tag_reten_molt
  p2 <- 1 - obs_G_prob * (1 - tag_reten_nomolt / tag_reten_molt)
  return(p1 / p2)
}
```

5. Implementing the Likelihood in RTMB

The likelihood function calculates joint negative log-likelihoods combining the Binomial process for molt occurrence and the Continuous process (Normal or Lognormal) for molt increments, integrated across length bin selectivity thresholds.

```
est_growth_sdt <- function(parms) {
  getAll(parms, data_sdt)

  n <- length(slength)
  X <- cbind(rep(1, n), slength, dal, temp)

  beta_logistic <- c(int_pg, beta_slength_pg, beta_dal_pg, beta_temp_pg)
  beta_linear   <- c(int_ag, beta_slength_ag, beta_dal_ag, beta_temp_ag)
  cv            <- exp(logcv)

  # Predict growth probability and increment
  Y_pg   <- X %*% beta_logistic
  G_prob <- exp(Y_pg) / (1 + exp(Y_pg))
  G_mean <- X %*% beta_linear

  if (dist == "normal")    G_sd <- G_mean * cv
  if (dist == "lognormal") G_sd <- rep(cv, n)

  # Numerical integration over selectivity bins
  b <- length(lbins)
  p_obs_integral   <- numeric(n)
  p_obs_integral_a <- numeric(n)

  for (i in 1:n) {
    selectivity <- sel(slength[i] + lbins, sel_int, sel_slope)
    phi1 <- pnorm(log(lbins[2]), log(G_mean[i]), G_sd[i])
    p_G_amt   <- numeric(b)
    p_G_amt_a <- numeric(b)

    p_G_amt[1]   <- (1 - G_prob[i]) * phi1
    p_G_amt_a[1] <- phi1
```

```
    for (j in 2:(b - 1)) {
      phi2 <- pnorm(log(lbins[j]), log(G_mean[i]), G_sd[i])
      p_G_amt[j]   <- G_prob[i] * (phi2 - phi1)
      p_G_amt_a[j] <- phi2 - phi1
      phi1 <- phi2
    }
    p_G_amt[b]   <- 1 - sum(p_G_amt[1:(b - 1)])
    p_G_amt_a[b] <- 1 - sum(p_G_amt_a[1:(b - 1)])

    p_obs_integral[i]   <- sum(selectivity * p_G_amt)
    p_obs_integral_a[i] <- sum(selectivity * p_G_amt_a)
  }

  # Likelihood 1: Binomial Molt Probability
  p_obs_G <- p_obs_integral * G_prob /
             (p_obs_integral * G_prob + sel(slength, sel_int, sel_slope) *
(1 - G_prob))
  nll_1   <- -sum(dbinom(grow_yn, size = 1, prob = p_obs_G, log = TRUE))

  # Likelihood 2: Lognormal Increment size for growers
  nll_2 <- 0
  for (i in 1:n) {
    if (grow_yn[i] == 1) {
      Sel <- sel(slength[i] + obs_inc[i], sel_int, sel_slope)
      nll_2 <- nll_2 - dnorm(log(obs_inc[i]), log(G_mean[i]), G_sd[i], log
= TRUE) -
               log(Sel) + log(p_obs_integral_a[i])
    }
  }

  negLL <- nll_1 + nll_2
  REPORT(negLL)
  return(negLL)
}
```

6. Constructing Growth Transition Matrices (GTMs)

Once parameters are estimated, continuous predictions are discretized into length-bin transition matrices spanning bins (e.g. 53 mm to 223 mm CL at 5 mm width).

```
make_trans_matrix <- function(G_prob, G_mean, G_sd, lengthbins, binwd) {
  nb <- length(lengthbins)
```

```
  Tmat <- matrix(0, nrow = nb, ncol = nb)
  diag(Tmat) <- 1 - G_prob

  for (i in 1:(nb - 2)) {
    Tmat[i, i] <- Tmat[i, i] + G_prob[i] * plnorm(binwd / 2,
log(G_mean[i]), G_sd[i])
    for (j in (i + 1):(nb - 1)) {
      phi1 <- plnorm(lengthbins[j] - lengthbins[i] - binwd / 2,
log(G_mean[i]), G_sd[i])
      phi2 <- plnorm(lengthbins[j + 1] - lengthbins[i] - binwd / 2,
log(G_mean[i]), G_sd[i])
      Tmat[i, j] <- G_prob[i] * (phi2 - phi1)
    }
    Tmat[i, nb] <- G_prob[i] * (1 - phi2)
  }
  Tmat[nb - 1, nb] <- G_prob[nb - 1]
  Tmat[nb, nb] <- 1
  return(Tmat)
}
```

7. Multi-Year Population Trajectory Simulation

The code projects an initial recruit pulse at 53 mm CL across annual or quarterly transitions, tracking length-at-age distributions and comparing against ASMFC 2025 benchmark assessment GTMs and an empirical age-at-length study (Huntsberger et al. 2020).

```
# Simulation logic for cohort transition
bins <- seq(53, 223, 5)
pr   <- 50
fN   <- matrix(0, nrow = pr, ncol = length(bins))
fN[1, 1] <- 100 # Initial recruit pulse

# Step forward through time using Matrix Multiplication
for (t in 1:(pr - 1)) {
  s2 <- fN[t, ] %*% estfGg[,,2] # Quarter / Season 2 GTM
  fN[t + 1, ] <- s2 %*% estfGg[,,3] # Quarter / Season 3 GTM
}

# Extract mean carapace length over time
midbins <- bins + 2.5
mean_length_at_age <- (fN / rowSums(fN)) %*% midbins
```

8. Output Summary & Best Practices

- **Convergence Check:** Verify optimization convergence via opt_sdt$convergence == 0 and ensure standard errors are positive definite via sdreport(obj_sdt).